

11. Highlight NaN Values

Aim

To create a DataFrame with random values, insert NaN values, and highlight the missing values.

Algorithm

1. Import pandas and numpy.
2. Create a DataFrame with random numbers.
3. Replace some values with NaN.
4. Use `style.highlight_null()` to highlight NaN values.
5. Display the DataFrame.

Code

```
import pandas as pd
import numpy as np

df = pd.DataFrame(np.random.randint(1,100,(10,4)),
                  columns=['A','B','C','D'])

df.iloc[1,2] = np.nan
df.iloc[4,0] = np.nan
df.iloc[7,3] = np.nan

print(df)

styled = df.style.highlight_null(color='red')

styled

output:
```

	A	B	C	D
0	9.000000	5	29.000000	22.000000
1	24.000000	50	nan	54.000000
2	25.000000	94	62.000000	90.000000
3	60.000000	52	23.000000	14.000000
4	nan	97	24.000000	19.000000
5	82.000000	47	23.000000	14.000000
6	7.000000	69	28.000000	15.000000
7	47.000000	60	51.000000	nan
8	85.000000	17	62.000000	52.000000
9	64.000000	25	60.000000	19.000000

12. Set Background Black and Font Yellow

Aim

To set the DataFrame background color to black and font color to yellow.

Algorithm

1. Import pandas and numpy.
2. Create a random DataFrame.
3. Apply style properties.
4. Display the styled DataFrame.

Code

```
import pandas as pd
import numpy as np
df = pd.DataFrame(np.random.randint(1,100,(10,4)),
                  columns=['A','B','C','D'])
styled = df.style.set_properties(**{
    'background-color':'black',
    'color':'yellow'
})
```

Styled

output:



	A	B	C	D
0	20	73	18	39
1	22	81	85	98
2	43	99	53	88
3	13	5	57	3
4	53	82	56	38
5	93	82	83	64
6	86	4	96	61
7	48	25	80	3
8	51	81	69	55
9	45	52	85	37

13. Detect Missing Values

Aim

To detect missing values in a DataFrame.

Algorithm

1. Create a DataFrame.
2. Insert NaN values.
3. Use `isnull()` to check missing values.
4. Display True or False.

Code

```
import pandas as pd
import numpy as np
```

```
df = pd.DataFrame({
    'A':[1,np.nan,3],
    'B':[4,5,np.nan],
    'C':[7,8,9]
})
```

```
print(df)
print(df.isnull())
```

output:

	A	B	C	D
0	55	6	75	31
1	22	38	63	88
2	36	67	46	25
3	49	94	64	11
4	93	77	98	68
5	67	49	37	79
6	52	46	7	42
7	69	36	98	6
8	94	40	71	13
9	81	35	91	20

14. Replace Missing Values

Aim

To replace missing values in a DataFrame.

Algorithm

1. Create a DataFrame.
2. Insert NaN values.
3. Replace NaN using fillna().
4. Display updated DataFrame.

Code

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'A':[1,np.nan,3],
    'B':[4,5,np.nan],
    'C':[7,8,9]
})
print("Original DataFrame")
print(df)
df = df.fillna(0)
print("Updated DataFrame")
```

```
print(df)
```

output:

```
... Original DataFrame
      A    B    C
0  1.0  4.0  7
1  NaN  5.0  8
2  3.0  NaN  9
Updated DataFrame
      A    B    C
0  1.0  4.0  7
1  0.0  5.0  8
2  3.0  0.0  9
```

15. Keep Rows Having At Least 2 NaN Values

Aim

To display rows containing at least two missing values.

Algorithm

1. Create a DataFrame.
2. Insert NaN values.
3. Count NaN values in each row.
4. Keep rows where count ≥ 2 .

Code

```
import pandas as pd
import numpy as np

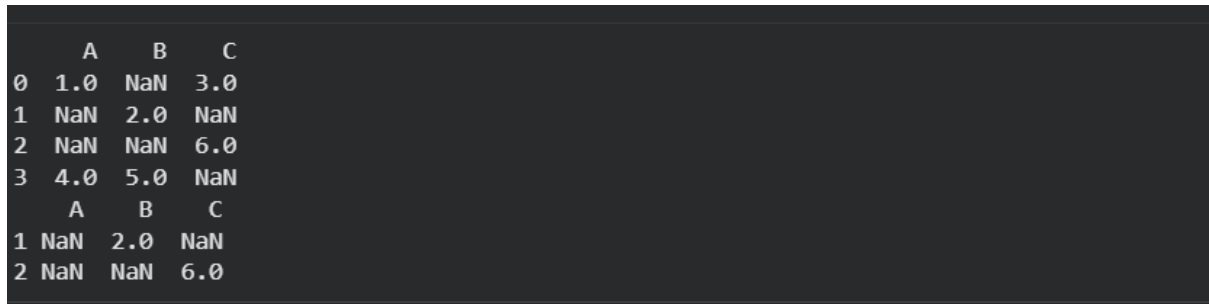
df = pd.DataFrame({
    'A': [1, np.nan, np.nan, 4],
    'B': [np.nan, 2, np.nan, 5],
    'C': [3, np.nan, 6, np.nan]
})

print(df)

result = df[df.isnull().sum(axis=1) >= 2]
```

```
print(result)
```

output:



	A	B	C
0	1.0	NaN	3.0
1	NaN	2.0	NaN
2	NaN	NaN	6.0
3	4.0	5.0	NaN

	A	B	C
1	NaN	2.0	NaN
2	NaN	NaN	6.0

16. Group by School Code

Aim

To split the DataFrame into groups based on school code.

Algorithm

1. Create DataFrame.
2. Group by School_Code.
3. Display groups.
4. Display GroupBy object type.

Code

```
import pandas as pd

df = pd.DataFrame({
    'School_Code':['S1','S1','S2','S2','S3'],
    'Name':['A','B','C','D','E'],
    'Age':[15,16,15,17,16]
})

group = df.groupby('School_Code')

print(type(group))

for name,data in group:
    print(name)
```

```
print(data)
```

output:

```
... <class 'pandas.core.groupby.generic.DataFrameGroupBy'>
S1
  School_Code Name  Age
0          S1    A   15
1          S1    B   16
S2
  School_Code Name  Age
2          S2    C   15
3          S2    D   17
S3
  School_Code Name  Age
4          S3    E   16
```

17. Mean, Minimum and Maximum Age

Aim

To calculate mean, minimum and maximum age for each school.

Algorithm

1. Create DataFrame.
2. Group by School_Code.
3. Apply mean(), min(), max().
4. Display result.

Code

```
import pandas as pd
df = pd.DataFrame({
    'School_Code':['S1','S1','S2','S2','S3'],
    'Age':[15,16,15,17,16]
})
result = df.groupby('School_Code')['Age'].agg(['mean','min','max'])
print(result)
```

output:

```

...
School_Code  mean  min  max
S1           15.5   15   16
S2           16.0   15   17
S3           16.0   16   16

```

18. Group by School Code and Class

Aim

To split the DataFrame using School Code and Class.

Algorithm

1. Create DataFrame.
2. Group using two columns.
3. Display each group.

Code

```

import pandas as pd

df = pd.DataFrame({
    'School_Code':['S1','S1','S2','S2','S3'],
    'Class':['10','10','9','9','8'],
    'Name':['A','B','C','D','E']
})

group = df.groupby(['School_Code','Class'])

for name,data in group:

```



```
print(name)
```

```
print(data)
```

output:

```
... ('S1', '10')
      School_Code Class Name
0         S1      10      A
1         S1      10      B
('S2', '9')
      School_Code Class Name
2         S2       9      C
3         S2       9      D
('S3', '8')
      School_Code Class Name
4         S3       8      E
```

19. Shape and Column Names of World Alcohol Dataset

Aim

To display the dimensions and column names of the World Alcohol Consumption dataset.

Algorithm

1. Import pandas.
2. Read CSV dataset.
3. Display shape.
4. Display column names.

Code

```
import pandas as pd
```

```
df = pd.read_csv("world_alcohol.csv")
```

```
print("Shape of DataFrame")
```

```
print(df.shape)
```

```
print("Column Names")
```

```
print(df.columns)
```

output

```
*** Shape of sample DataFrame
(3000, 9)
Column Names of sample DataFrame
Index(['longitude', 'latitude', 'housing_median_age', 'total_rooms',
      'total_bedrooms', 'population', 'households', 'median_income',
      'median_house_value'],
      dtype='object')
First 5 rows of sample DataFrame
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income
0	-122.05	37.37	27.0	3885.0	661.0	1537.0	606.0	
1	-118.30	34.26	43.0	1510.0	310.0	809.0	277.0	
2	-117.81	33.78	27.0	3589.0	507.0	1484.0	495.0	
3	-118.36	33.82	28.0	67.0	15.0	49.0	11.0	
4	-119.67	36.33	19.0	1241.0	244.0	850.0	237.0	

20. Find Index of a Substring in a DataFrame Column

Aim

To find the index position of a substring in a DataFrame column.

Algorithm

1. Create DataFrame.
2. Use str.find() to locate substring.
3. Display index positions.

Code

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Apple', 'Banana', 'Mango', 'Orange']
})

df['Index'] = df['Name'].str.find('an')

print(df)
```

output:

	Name	Index
0	Apple	-1
1	Banana	1
2	Mango	1
3	Orange	2